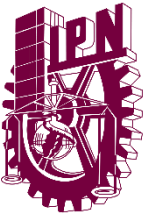




**Unidad Profesional Interdisciplinaria de Ingeniería y Ciencias
Sociales y Administrativas.**



Reporte de prácticas Abstracción y uso de datos Segundo Parcial

**NOMBRE DE LOS INTEGRANTES:
FUENTES HERNANDEZ MIGUEL ANGEL
RAMIREZ REYES LORENA AMEYALLI
TELLEZ GIRON PATLANI DAVID**

3CM32

ABSTRACCIÓN Y USO DE DATOS

Contenido

Práctica 10 (primer parcial).....	3
Práctica 13 (primer parcial).....	5
Práctica 1	6
Práctica 3	9
Práctica 4	11
Práctica 5	13
Practica No. 16	15
Practica No. 17	17
Practica No. 18	19
Practica No. 19	21
Practica No. 20	23
Practica No. 21	25
Práctica 6 – Pila Dinámica	27
Práctica 7– Cola Dinámica.....	28
Práctica 8– Pila	29
Práctica 9 – Cola	30
Práctica 10 – Lista	31
Práctica 7 Bis.....	32

Práctica 10 (primer parcial)

Este código implementa un sistema de procesamiento de datos numéricos bajo el paradigma de Programación Orientada a Objetos en C++, donde la característica principal es el uso de un puntero para gestionar un arreglo de forma indirecta. Al iniciar el programa, el constructor de la clase vincula un puntero a la dirección base de un arreglo privado, permitiendo que métodos como la captura de datos y el cálculo de estadísticas (suma, promedio, máximo y mínimo) operen directamente sobre las direcciones de memoria mediante aritmética de punteros en lugar de usar subíndices tradicionales. Esta estructura garantiza el encapsulamiento de la información al mantener los datos protegidos dentro de la clase, mientras que la lógica de comparación asegura que el ciclo recorra la totalidad del bloque de memoria antes de retornar los resultados finales al programa principal.

1. Definición de la Clase: Se define la clase y se colocan las variables en la sección private. Necesitas el arreglo (donde se guardan los datos) y el puntero (que servirá para el acceso indirecto).

```
class Promedio{  
private:  
    float numeros[5];  
    float* ptr;
```

2. Inicialización (Constructor): Se crea el constructor de la clase para asignar la dirección de memoria del arreglo al puntero. Esto asegura que el puntero no esté "volando" y sepa dónde empieza la información.

```
public:  
    Promedio(){  
        ptr= numeros;  
    }
```

3. Captura de Datos Indirecta: Se crea un método (como Datos()) que use un ciclo for. Dentro, en lugar de usar subíndices, se usa la aritmética de punteros para guardar los valores que el usuario ingresa.

```
void Datos(){  
    cout << "Ingresa 5 números: " << endl;  
    for (int i= 0; i<5; i++){  
        cout << "Número " << i+1 << ": ";  
        cin >> *(ptr + i);  
    }  
}
```

4. Procesamiento de Información: Se crean los métodos para los cálculos (Suma, Maximo, Minimo). En cada uno, se recorre el arreglo usando el puntero.

Regla de oro: El return del resultado siempre debe ir después de que el ciclo for haya terminado de revisar todos los números.

```
float Suma() {  
    float suma= 0;  
    for(int i= 0; i<5; i++) suma += *(ptr + i);  
    return suma;  
}  
  
float Maximo() {  
    float maximo= *ptr;  
    for (int i= 1; i<5; i++) {  
        if (*(ptr + i) > maximo) {  
            maximo = *(ptr + i);  
        }  
    }  
    return maximo;  
}  
  
float Minimo() {  
    float minimo= *ptr;  
    for (int i= 1; i<5; i++) {  
        if (*(ptr + i) < minimo) {  
            minimo = *(ptr + i);  
        }  
    }  
    return minimo;  
}
```

5. Instanciación y Llamada: En la función main, se crea el objeto (instancia de la clase) y se llaman los métodos en orden lógico: primero el de entrada de datos y al final el de mostrar resultados.

```
Promedio miPuntero;  
  
miPuntero.Datos();  
miPuntero.Resultados();
```

Práctica 13 (primer parcial)

Este código implementa un sistema de gestión de datos personales bajo el paradigma de POO en C++, donde la característica principal es el uso de un puntero para gestionar un arreglo de forma indirecta. Al iniciar el programa, el constructor de la clase vincula un puntero a la dirección base de un bloque de memoria dinámico, permitiendo que métodos como la captura y la impresión de datos operen directamente sobre las direcciones de memoria en lugar de usar solo variables locales estáticas. Esta estructura garantiza el encapsulamiento al mantener los datos protegidos dentro de la clase.

1. Se define la clase y se declaran las variables en la sección private: Se necesita el puntero, que servirá para el acceso indirecto a la lista de personas, y una variable para el tamaño del arreglo.
2. El constructor de la clase: Su función es asignar memoria dinámica mediante new al puntero. Esto asegura que el puntero no esté "volando" y sepa exactamente dónde empieza la información en la memoria.
3. El Destructor: Se utiliza para liberar la memoria del arreglo con la instrucción delete[] lista;. Esto evita que la memoria se quede bloqueada una vez que el objeto deja de existir.
4. Se crean los métodos de Interfaz:
 - Captura: Un método que use un ciclo for para pedir los datos (nombre, apellidos, edad) y guardarlos en la posición del arreglo apuntada.
 - Mostrar: Un método que recorra el arreglo usando el puntero para imprimir la lista completa en pantalla.
5. Función main: Se crea el objeto (instancia de la clase) pasando el tamaño deseado y se llaman los métodos en orden lógico: primero el de entrada de datos y al final el de mostrar resultados.

```
class Persona {
private:
    string nombre, ap, am, genero;
    int edad;

public:
    void leerDatos() {
        cout << "Nombre: "; cin >> nombre;
        cout << "Apellido Paterno: "; cin >> ap;
        cout << "Apellido Materno: "; cin >> am;
        cout << "Genero: "; cin >> genero;
        cout << "Edad: "; cin >> edad;
        cout << endl;
    }

    void mostrarDatos() {
        cout << nombre << " " << ap << " " << am
            << " | " << genero << " | " << edad << " años" << endl;
    }
};
```

```
class Personas {
private:
    Persona* lista;
    int total;

public:
    Personas(int n) {
        total = n;
        lista = new Persona[total];
    }

    void misDatos(n);

    void capturar();
    void imprimir();
};
```

Práctica 1

Este reporte técnico describe la implementación de un sistema de gestión de información bajo el paradigma de Programación Orientada a Objetos (POO), evolucionando los códigos base de Practica12ListaPersona.cpp y Practica12ListaAutos.cpp hacia un modelo de gestión de memoria dinámica mediante acceso indirecto.

1. Definición de la Infraestructura: Contenedor.h

El núcleo del proyecto reside en este archivo de encabezado, donde se definen los "nuevos tipos de dato" que encapsulan la lógica de punteros. El reporte destaca tres secciones fundamentales:

Declaración de Atributos Privados: En lugar de declarar arreglos estáticos, las clases gestoras como ListaPersonasIndirecta y ListaAutosIndirecta contienen un puntero al tipo de dato base y una variable de control, por ejemplo: Persona* lista; y int tamano;. Esto garantiza el encapsulamiento, impidiendo que el programa principal manipule la memoria directamente.

```
class ListaPersonasIndirecta {  
private:  
    Persona* lista;  
    int tamano;
```

```
class ListaAutosIndirecta {  
private:  
    Auto* listaAutos;  
    int cant;
```

Gestión Dinámica de Memoria: El constructor de la clase es el encargado de dar vida al arreglo mediante la instrucción lista = new Persona[tamano];. Esta acción reserva un bloque de memoria en el *heap*, permitiendo que el tamaño de la lista sea flexible. Por otro lado, siguiendo la regla de oro de la gestión de recursos, el destructor utiliza delete[] lista; para liberar dicha memoria automáticamente cuando el objeto sale de su ámbito, evitando fugas de memoria.

```
public:  
    ListaPersonasIndirecta(int n) {  
        tamano = n;  
        lista = new Persona[tamano];  
        cout << "(Objeto Gestor: Memoria dinámica reservada)" << endl;  
    }  
  
    ~ListaPersonasIndirecta() {  
        delete[] lista;  
        cout << "(Objeto Gestor: Memoria dinámica liberada correctamente)" << endl;  
    }
```

```
public:  
    ListaAutosIndirecta(int n) {  
        cant = n;  
        listaAutos = new Auto[cant];  
    }  
  
    ~ListaAutosIndirecta() {  
        delete[] listaAutos;  
    }
```

Métodos de Interfaz: Se implementan funciones como `capturarLista()` e `imprimirReporte()` que utilizan ciclos para recorrer el arreglo a través del puntero. Estas funciones abstraen la complejidad técnica, permitiendo que el usuario interactúe con los datos de forma sencilla.

```
void capturarLista() {
    for (int i = 0; i < tamano; i++) {
        cout << "--- Persona " << i + 1 << " ---" << endl;
        lista[i].leerDatos();
    }
}

void mostrarReporte() {
    cout << "\n--- LISTA DE PERSONAS REGISTRADAS ---" << endl;
    for (int i = 0; i < tamano; i++) {
        lista[i].mostrarDatos();
    }
}

void capturarAutos() {
    for (int i = 0; i < cant; i++) {
        cout << "--- Auto " << i + 1 << " ---" << endl;
        listaAutos[i].leerDatos();
    }
}

void imprimirInventario() {
    cout << "\n--- INVENTARIO DE AUTOS ---" << endl;
    for (int i = 0; i < cant; i++) {
        listaAutos[i].mostrarDatos();
    }
}
```

2. Implementación Lógica: ContPersona.cpp

Este archivo actúa como el controlador para la entidad Persona. El reporte identifica que su función principal es instanciar el nuevo tipo de dato mediante la línea `ListaPersonasIndirecta miRegistro(cantidad);`.

```
ListaPersonasIndirecta miRegistro(cantidad);
miRegistro.capturarLista();
miRegistro.mostrarReporte();
```

Al realizar esta declaración, el programa no crea una lista de personas en la pila (stack), sino que crea un objeto gestor que "apunta" hacia la lista real en otra sección de la memoria. La ejecución sigue un orden lógico estrictamente definido: primero se invoca el método de captura para poblar el arreglo dinámico y, posteriormente, se solicita la impresión de los datos procesados, delegando toda la responsabilidad del manejo de punteros al objeto creado.

3. Implementación Lógica: ContAutos.cpp

De manera análoga, este archivo aplica los mismos principios de POO para la entidad Auto. El reporte subraya la reutilización de código y la modularidad, ya que este programa utiliza el mismo archivo de encabezado para gestionar un tipo de dato completamente distinto.

La instrucción `ListaAutosIndirecta inventario(n);` demuestra cómo el acceso indirecto permite manejar estructuras de datos complejas (con cadenas de texto y flotantes) de forma organizada. Al igual que en el programa de personas, el flujo de datos se mantiene protegido dentro del objeto inventario, confirmando que el acceso a la lista de autos es indirecto, eficiente y seguro bajo los estándares de la programación orientada a objetos actual.

```
ListaAutosIndirecta inventario(n);  
  
inventario.capturarAutos();  
inventario.imprimirInventario();
```


Práctica 3

Esta práctica muestra la implementación de las Estructuras de Datos Lineales (Pila, Cola y Lista) bajo el paradigma de Programación Orientada a Objetos (POO) en el lenguaje Java. El objetivo principal es demostrar el manejo de memoria estática mediante el uso de arreglos de tamaño fijo, subrayando la importancia de las validaciones de seguridad para prevenir errores comunes en tiempo de ejecución.

A través de este ejercicio, se evidencia cómo el encapsulamiento permite aislar la lógica interna de cada estructura (como el puntero top en la Pila o el desplazamiento de elementos en la Cola) de la interacción con el usuario. Asimismo, la práctica destaca la importancia de los métodos de control (estaVacía y estaLlena) como mecanismos críticos para garantizar la integridad de los datos y la estabilidad del sistema ante condiciones de desbordamiento (*Overflow*) o vaciado (*Underflow*).

1. Estructura de Memoria y Atributos

La pila se implementa de manera estática, lo que significa que su tamaño es fijo y se reserva en memoria desde el momento de su creación.

- Arreglo (int[]): Es el contenedor físico donde se almacenan los datos de forma contigua.
- Capacidad: Variable que define el límite máximo de elementos para evitar desbordamientos de memoria.
- Tope (top): Es un índice entero que actúa como puntero. Inicia en -1 para indicar que la pila está vacía y se desplaza conforme se agregan o eliminan elementos.

```
public class Pila_estatica {  
    private int top;  
    private int capacidad;  
    private int[] arreglo;
```

2. Principio de Operación (LIFO)

El funcionamiento se rige por la regla de que el último elemento en entrar es el primero en salir. No se puede acceder a elementos en el fondo sin haber retirado primero los que están encima.

3. Operaciones Fundamentales y Validaciones

Para que el código sea robusto, cada operación debe validar el estado de la pila antes de proceder:

- Push (Apilar):
 - Validación: Primero verifica si la pila está llena (`top == capacidad - 1`). Si lo está, ocurre un Stack Overflow.
 - Acción: Si hay espacio, el top aumenta en 1 y el dato se guarda en esa nueva posición del arreglo.
- Pop (Desapilar):
 - Validación: Verifica si la pila está vacía (`top == -1`). Si lo está, ocurre un Stack Underflow.
 - Acción: Se devuelve el valor almacenado en la posición actual de top y luego el índice disminuye en 1, "olvidando" el elemento anterior.

```
public void push(int dato) {
    if (!estaLlena()) {
        arreglo[++top] = dato;
        System.out.println("-> Insertado con éxito.");
    } else {
        System.out.println("Error: Stack Overflow (Pila llena)");
    }
}

public void pop() {
    if (!estaVacia()) {
        System.out.println("-> Sacado: " + arreglo[top--]);
    } else {
        System.out.println("Error: Stack Underflow (Pila vacía)");
    }
}
```

4. Gestión de Errores

El uso de métodos booleanos como `estaVacia()` y `estaLlena()` es esencial en POO. Estos métodos encapsulan la lógica de control, permitiendo que el programa principal tome decisiones antes de intentar manipular datos inexistentes o espacios agotados, garantizando la estabilidad de la aplicación.

```
public boolean estaVacia() { return top == -1; }
public boolean estaLlena() { return top == capacidad - 1; }
```

Práctica 4

Durante el desarrollo de la práctica, se explora el uso de memoria estática, empleando arreglos de tamaño definido para simular comportamientos lógicos específicos. Se pone especial énfasis en el control de flujos mediante los principios LIFO (Last In, First Out) para la Pila y FIFO (First In, First Out) para la Cola, además de la manipulación por índices en la Lista.

A diferencia de un enfoque procedimental simple, esta práctica muestra cómo el encapsulamiento permite proteger la integridad de los datos, obligando a que toda interacción con la estructura pase por métodos validados que previenen errores críticos como el desbordamiento (*Overflow*) o el acceso a posiciones inexistentes (*Underflow*). Esta aproximación no solo refuerza el aprendizaje de las estructuras de datos, sino también las buenas prácticas de ingeniería de software y robustez de código.

1. Definición y Estructura

La cola es una estructura de datos lineal que opera bajo el principio FIFO (First In, First Out), donde el primer elemento en ser almacenado es el primero en ser retirado.

- Contenedor: Se utiliza un arreglo unidimensional de tamaño fijo definido durante la instanciación del objeto.
- Puntero Final (fin): Se utiliza un índice que marca la posición del último elemento insertado. Inicia en -1 y se incrementa con cada inserción.

```
public class Cola_estatica {  
    private int fin;  
    private int capacidad;  
    private int[] arreglo;  
  
    public Cola_estatica(int tamano) {  
        this.capacidad = tamano;  
        this.arreglo = new int[capacidad];  
        this.fin = -1;  
    }  
}
```

2. Dinámica de Operación (FIFO)

A diferencia de la pila, la cola simula una fila de espera real. La inserción se realiza siempre por el final, mientras que la extracción se realiza siempre desde el frente (posición 0).

3. Métodos Críticos y Desplazamiento

Para mantener la integridad de la estructura en un entorno estático, se implementan las siguientes funciones:

- Enqueue (Encolar):
 - Validación: Se invoca el método `estaLlena()`. Si el índice `fin` es igual a `capacidad - 1`, se impide la entrada de nuevos datos para evitar errores de memoria.
 - Acción: Se aumenta el índice `fin` y se coloca el nuevo dato en esa posición.
- Dequeue (Desencolar):
 - Validación: Se invoca el método `estaVacia()`. Si `fin` es `-1`, no hay elementos que procesar.
 - Reordenamiento (Shift): Esta es la parte más importante del código estático. Al retirar el elemento en la posición `0`, todos los elementos restantes deben desplazarse una posición a la izquierda mediante un ciclo `for`. Esto garantiza que el siguiente elemento en la fila siempre ocupe el frente de la cola.

```
public void enqueue(int dato) {
    if (!estaLlena()) {
        arreglo[++fin] = dato;
        System.out.println("-> Encolado correctamente.");
    } else {
        System.out.println("Error: Cola llena.");
    }
}

public void dequeue () {
    if (!estaVacia()) {
        System.out.println("-> Desencolado: " + arreglo[0]);
        for (int i = 0; i < fin; i++) arreglo[i] = arreglo[i + 1];
        fin--;
    } else {
        System.out.println("Error: Cola vacía.");
    }
}
```

Práctica 5

En esta práctica se muestra la implementación de una lista basada en arreglos, donde el control no depende de un orden de entrada (como LIFO o FIFO), sino de la ubicación lógica de los datos. Se demuestra cómo manipular la memoria contigua para realizar inserciones y borrados quirúrgicos, manteniendo siempre la cohesión de la información a través de algoritmos de desplazamiento manual de elementos.

1. Concepto y Estructura

A diferencia de la pila y la cola, la lista estática permite insertar y eliminar elementos en cualquier posición del arreglo, siempre que esta se encuentre dentro del rango de datos actuales.

- Arreglo: Almacén de datos de tamaño fijo.
- Tamaño Actual (tamanoActual): Contador que indica cuántos elementos reales hay en la lista. Sirve para controlar el límite de las iteraciones y validar las posiciones de inserción.

```
public class Lista_estatica {  
    private int[] arreglo;  
    private int tamanoActual, capacidad;  
  
    public Lista_estatica(int cap) {  
        this.capacidad = cap;  
        this.arreglo = new int[cap];  
        this.tamanoActual = 0;  
    }  
}
```

2. Dinámica de Operación: Inserción por Índice

La operación de inserción no es directa si se realiza en medio de la lista.

- Validación: Se verifica que la lista no esté llena y que el índice solicitado sea mayor o igual a 0 y menor o igual al tamanoActual.
- Desplazamiento a la Derecha: Para no sobrescribir datos, todos los elementos desde la posición deseada hasta el final deben "moverse" un lugar a la derecha. Una vez abierto el hueco, se deposita el nuevo dato.

```
public void insertar(int dato, int pos) {  
    if (tamanoActual < capacidad && pos >= 0 && pos <= tamanoActual) {  
        for (int i = tamanoActual; i > pos; i--) arreglo[i] = arreglo[i - 1];  
        arreglo[pos] = dato;  
        tamanoActual++;  
        System.out.println("-> Insertado.");  
    } else System.out.println("Error: Posición inválida o lista llena.");  
}
```

3. Dinámica de Operación: Eliminación por Índice

- Validación: Se comprueba que la lista no esté vacía y que la posición exista dentro del rango de elementos ocupados.
- Desplazamiento a la Izquierda: Al borrar un dato, queda un "hueco" en el arreglo. Para mantener la integridad de la lista, todos los elementos a la derecha del eliminado se recorren un lugar a la izquierda para cerrar el espacio.

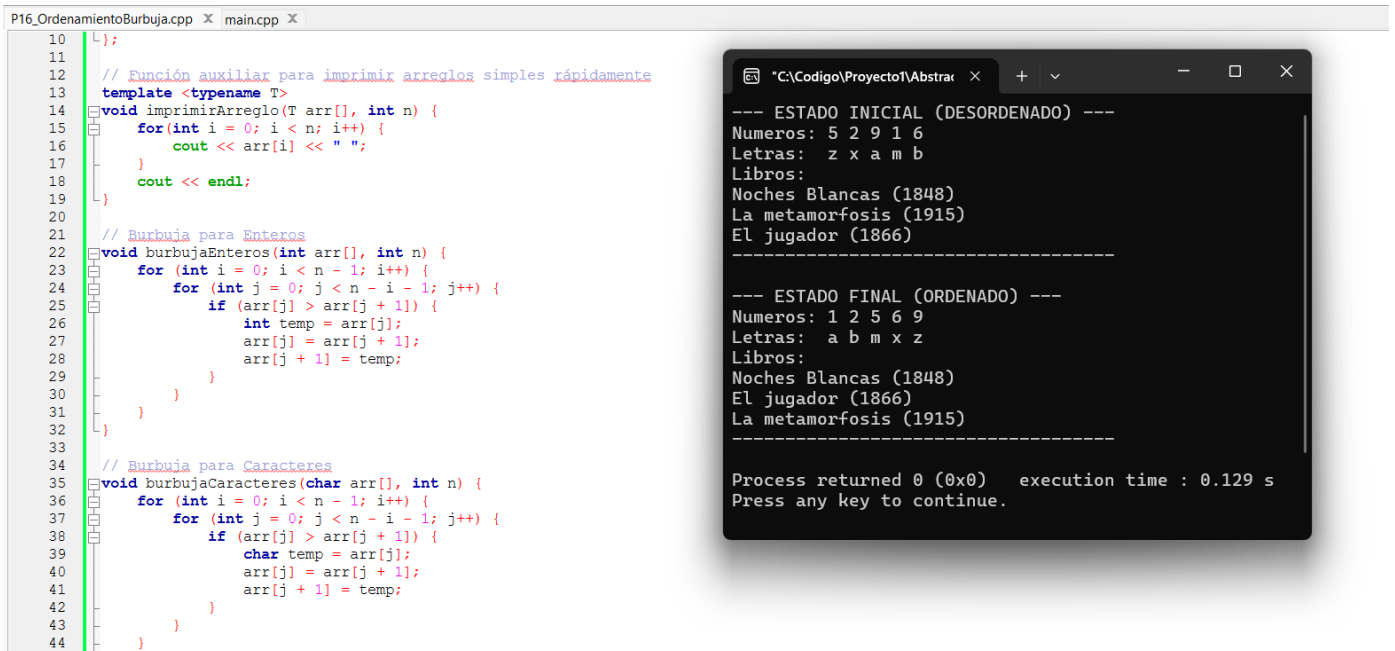
```
public void eliminar(int pos) {  
    if (pos >= 0 && pos < tamanoActual) {  
        for (int i = pos; i < tamanoActual - 1; i++) arreglo[i] = arreglo[i + 1];  
        tamanoActual--;  
        System.out.println("-> Eliminado.");  
    } else System.out.println("Error: Posición no válida.");  
}
```

Practica No. 16

En esta práctica vamos a acomodar 3 tipos de datos, caracteres, cadenas de texto y enteros, estos datos les vamos a dar definidos en el programa para ejemplificar el ordenamiento de burbuja. En este ordenamiento debemos saber que tamaño de arreglo utilizamos para poder determinar cuántas veces evaluaremos el número, carácter y año de los datos que ingresemos.

1. Primero determinamos la variable “i” para recorrer todos los espacios del arreglo
2. Declaramos otra variable: “j”, para acomodar los números (cuando encuentra el primer número lo evalúa, si el número es menor al primero que toma lo coloca primero
3. Cuando termina de evaluar todos los números guarda el arreglo para poder mostrarlo en pantalla

Esto es la definición del por qué se le llama ordenamiento burbuja, una empuja a otra hacia arriba para que se acomoden poco a poco, este tipo de ordenamiento es básico, no se recomienda que se utilice con grandes cantidades de datos por el tiempo y la capacidad de procesamiento que necesita.



The image shows a C++ program in a code editor and its execution output in a terminal window.

Code Editor (P16_OrdenamientoBurbuja.cpp):

```
10 //;
11
12 // Función auxiliar para imprimir arreglos simples rápidamente
13 template <typename T>
14 void imprimirArreglo(T arr[], int n) {
15     for(int i = 0; i < n; i++) {
16         cout << arr[i] << " ";
17     }
18     cout << endl;
19 }
20
21 // Burbuja para Enteros
22 void burbujaEnteros(int arr[], int n) {
23     for (int i = 0; i < n - 1; i++) {
24         for (int j = 0; j < n - i - 1; j++) {
25             if (arr[j] > arr[j + 1]) {
26                 int temp = arr[j];
27                 arr[j] = arr[j + 1];
28                 arr[j + 1] = temp;
29             }
30         }
31     }
32 }
33
34 // Burbuja para Caracteres
35 void burbujaCaracteres(char arr[], int n) {
36     for (int i = 0; i < n - 1; i++) {
37         for (int j = 0; j < n - i - 1; j++) {
38             if (arr[j] > arr[j + 1]) {
39                 char temp = arr[j];
40                 arr[j] = arr[j + 1];
41                 arr[j + 1] = temp;
42             }
43         }
44     }
45 }
```

Terminal Output:

```
--- ESTADO INICIAL (DESORDENADO) ---
Numeros: 5 2 9 1 6
Letras:  z x a m b
Libros:
Noches Blancas (1848)
La metamorfosis (1915)
El jugador (1866)
-----

--- ESTADO FINAL (ORDENADO) ---
Numeros: 1 2 5 6 9
Letras:  a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)
-----

Process returned 0 (0x0)   execution time : 0.129 s
Press any key to continue.
```

También vamos a dar un ejemplo de como funciona con estructura orientada a objetos, casi no cambiamos nada solo utilizamos un método dentro de una clase y llamamos a ese método con un objeto en la clase principal pero aquí utilizamos polimorfismo y la encapsulación, el primero lo vemos en el código cuando utilizamos un solo método para los tres casos y la encapsulación para crear un nuevo tipo de dato dentro de Programación Orientada a Objetos.

The image shows a C++ IDE with two windows. The left window displays the source code for a bubble sort algorithm using a generic template. The right window shows the output of the program, which includes a menu, initial state, and final state of the data being sorted.

```

7 class Libro {
8 private:
9     string titulo;
10    int anio;
11
12 public:
13     Libro() : titulo(""), anio(0) {} // Constructor por defecto
14     Libro(string t, int an) : titulo(t), anio(an) {} // Constructor
15
16     // Sobrecarga para comparar objetos
17     bool operator>(const Libro& otro) const {
18         return this->anio > otro.anio;
19     }
20
21     void mostrar() const {
22         cout << titulo << " (" << anio << ")" << endl;
23     }
24 };
25
26 // Clase con el algoritmo genérico
27 class Ordenador {
28 public:
29     // Plantilla que permite ordenar cualquier tipo de dato
30     template <typename T>
31     void ordenarBurbuja(T arr[], int n) {
32         for (int i = 0; i < n - 1; i++) {
33             for (int j = 0; j < n - i - 1; j++) {
34                 if (arr[j] > arr[j + 1]) {
35                     T temp = arr[j];
36                     arr[j] = arr[j + 1];
37                     arr[j + 1] = temp;
38                 }
39             }
40         }
41     }
42 };
43

```

```

--- MENU DE EJERCICIOS ---
1. Burbuja Estructurada (PE)
2. Burbuja Orientada a Objetos (POO)
Elige una opcion: 2
--- ESTADO INICIAL (DESORDENADO) ---
Numeros: 5 2 9 1 6
Letras:  z x a m b
Libros:
Noches Blancas (1848)
La metamorfosis (1915)
El jugador (1866)
-----

--- ESTADO FINAL (ORDENADO) ---
Numeros: 1 2 5 6 9
Letras:  a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)
-----

Process returned 0 (0x0)   execution time : 4.639 s
Press any key to continue.

```

Tuvimos que implementar un menú en el main para que no estemos borrándolo cada vez para usarlo en cada programa

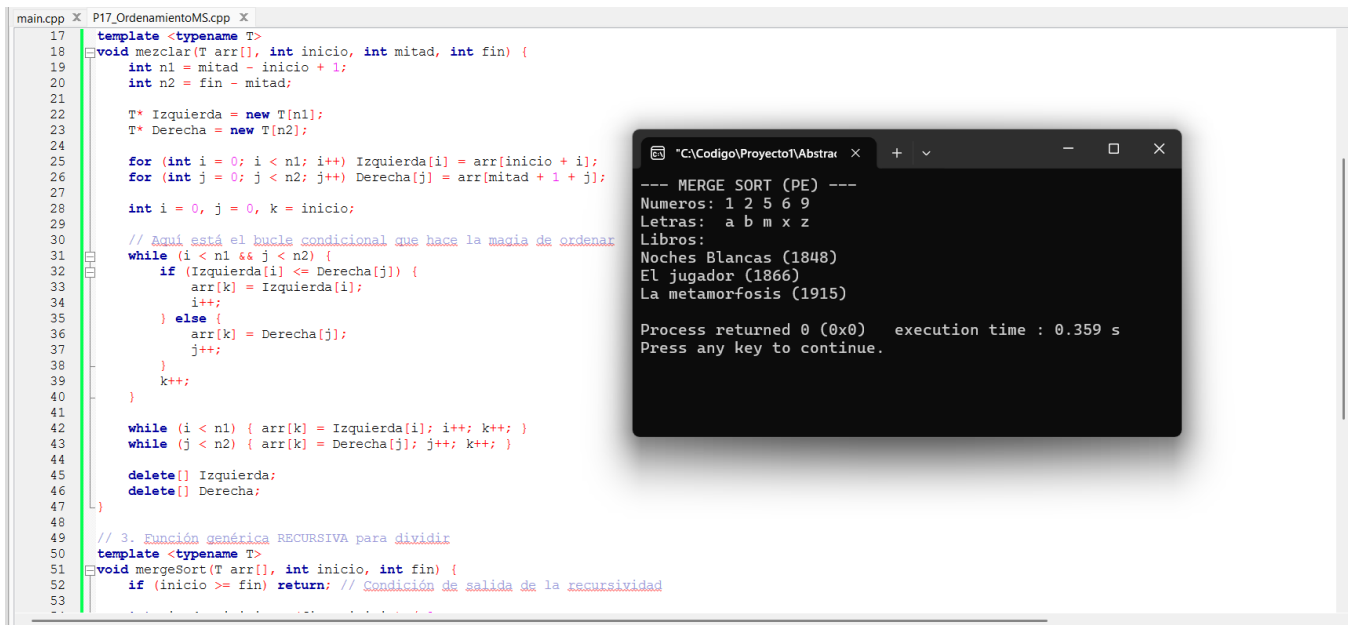
Practica No. 17

En esta práctica vamos a acomodar 3 tipos de datos: caracteres, enteros y un nuevo tipo de dato, utilizando el algoritmo de ordenamiento Merge Sort. Al igual que en la práctica anterior, los datos los damos definidos en el programa para ejemplificar su funcionamiento.

Para este ordenamiento utilizamos la estrategia conocida como "Divide y Vencerás". Primero, implementamos una función recursiva que se encarga de dividir el arreglo a la mitad una y otra vez. Esto lo hace hasta que el arreglo queda fragmentado en elementos individuales, basándose en la lógica de que un solo elemento ya se encuentra "ordenado".

Una vez dividido, utilizamos una segunda función para fusionar o mezclar estas mitades. Aquí declaramos un bucle con condiciones y utilizamos contadores que van evaluando el primer elemento de la mitad izquierda contra el de la mitad derecha. Si el elemento evaluado es menor o igual, lo coloca primero en el arreglo original y avanza al siguiente.

A diferencia del método de burbuja, el Merge Sort no empuja los números uno por uno, sino que une bloques ya ordenados. Esto hace que sea un algoritmo muchísimo más rápido, eficiente y altamente recomendado para procesar grandes cantidades de datos, ya que reduce drásticamente el tiempo de ejecución.



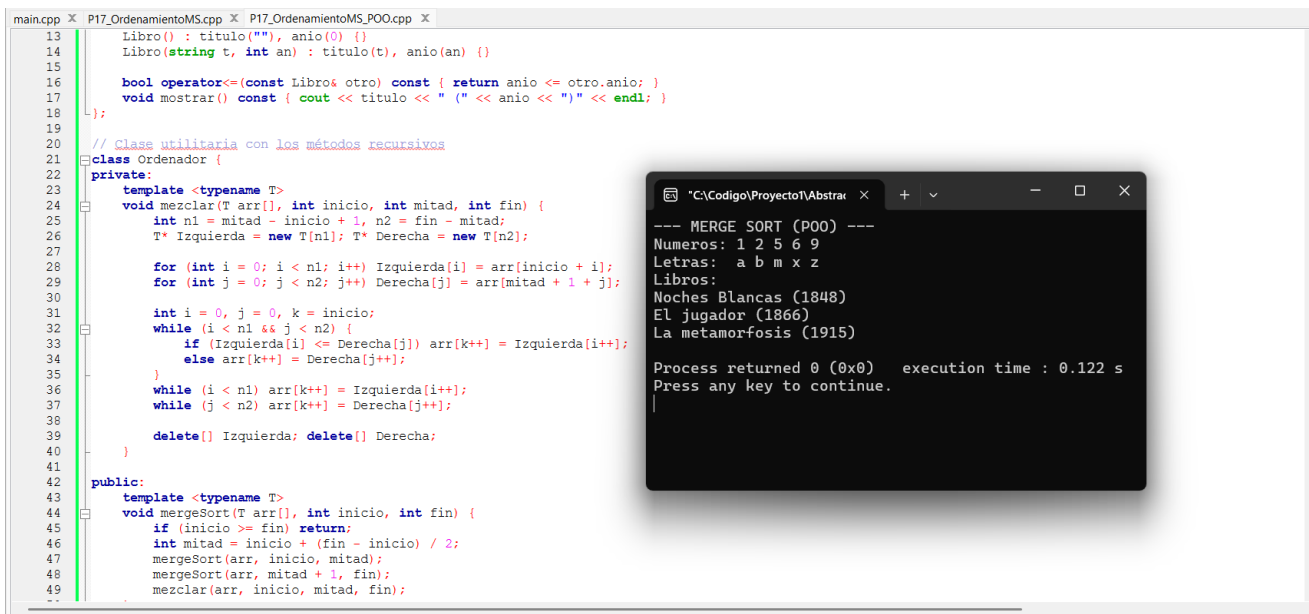
```
main.cpp x P17_OrdenamientoMS.cpp x
17 template <typename T>
18 void mezclar(T arr[], int inicio, int mitad, int fin) {
19     int n1 = mitad - inicio + 1;
20     int n2 = fin - mitad;
21
22     T* Izquierda = new T[n1];
23     T* Derecha = new T[n2];
24
25     for (int i = 0; i < n1; i++) Izquierda[i] = arr[inicio + i];
26     for (int j = 0; j < n2; j++) Derecha[j] = arr[mitad + 1 + j];
27
28     int i = 0, j = 0, k = inicio;
29
30     // Aquí está el bucle condicional que hace la magia de ordenar
31     while (i < n1 && j < n2) {
32         if (Izquierda[i] <= Derecha[j]) {
33             arr[k] = Izquierda[i];
34             i++;
35         } else {
36             arr[k] = Derecha[j];
37             j++;
38         }
39         k++;
40     }
41
42     while (i < n1) { arr[k] = Izquierda[i]; i++; k++; }
43     while (j < n2) { arr[k] = Derecha[j]; j++; k++; }
44
45     delete[] Izquierda;
46     delete[] Derecha;
47 }
48
49 // 3. Función genérica RECURSIVA para dividir
50 template <typename T>
51 void mergeSort(T arr[], int inicio, int fin) {
52     if (inicio >= fin) return; // Condición de salida de la recursividad
53     int mitad = (inicio + fin) / 2;
54     mergeSort(arr, inicio, mitad);
55     mergeSort(arr, mitad + 1, fin);
56     mezclar(arr, inicio, mitad, fin);
57 }
```

```
--- MERGE SORT (PE) ---
Numeros: 1 2 5 6 9
Letras:  a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)

Process returned 0 (0x0)   execution time : 0.359 s
Press any key to continue.
```

También demostramos su funcionamiento en Programación Estructurada y Orientada a Objetos. En ambos casos utilizamos plantillas para que un solo método pudiera ordenar los tres tipos de datos. Para el nuevo tipo de dato encapsulado en una clase, aplicamos la sobrecarga del operador `<=` para enseñarle al algoritmo cómo debe evaluar y comparar los años de los libros.

Finalmente, integramos este nuevo método de ordenamiento (tanto en su versión estructurada como orientada a objetos) como opciones adicionales dentro del menú de nuestro archivo main principal, mandándolos llamar como funciones independientes.



```
main.cpp x P17_OrdenamientoMS.cpp x P17_OrdenamientoMS_POO.cpp x
13  Libro() : titulo(""), anio(0) {}
14  Libro(string t, int an) : titulo(t), anio(an) {}
15
16  bool operator<=(const Libro& otro) const { return anio <= otro.anio; }
17  void mostrar() const { cout << titulo << " (" << anio << ")" << endl; }
18
19
20  // Clase utilitaria con los métodos recursivos
21  class Ordenador {
22  private:
23      template <typename T>
24      void mezclar(T arr[], int inicio, int mitad, int fin) {
25          int n1 = mitad - inicio + 1, n2 = fin - mitad;
26          T* Izquierda = new T[n1]; T* Derecha = new T[n2];
27
28          for (int i = 0; i < n1; i++) Izquierda[i] = arr[inicio + i];
29          for (int j = 0; j < n2; j++) Derecha[j] = arr[mitad + 1 + j];
30
31          int i = 0, j = 0, k = inicio;
32          while (i < n1 && j < n2) {
33              if (Izquierda[i] <= Derecha[j]) arr[k++] = Izquierda[i++];
34              else arr[k++] = Derecha[j++];
35          }
36          while (i < n1) arr[k++] = Izquierda[i++];
37          while (j < n2) arr[k++] = Derecha[j++];
38
39          delete[] Izquierda; delete[] Derecha;
40      }
41
42  public:
43      template <typename T>
44      void mergeSort(T arr[], int inicio, int fin) {
45          if (inicio >= fin) return;
46          int mitad = inicio + (fin - inicio) / 2;
47          mergeSort(arr, inicio, mitad);
48          mergeSort(arr, mitad + 1, fin);
49          mezclar(arr, inicio, mitad, fin);
50      }
51  };
52
53  int main() {
54      // ... (previous code) ...
55      Ordenador o;
56      o.mergeSort(arr, 0, n-1);
57      // ... (previous code) ...
58      return 0;
59  }
```

```
"C:\Codigo\Proyecto1\Abstrac x + v - □ x
--- MERGE SORT (POO) ---
Numeros: 1 2 5 6 9
Letras:  a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)

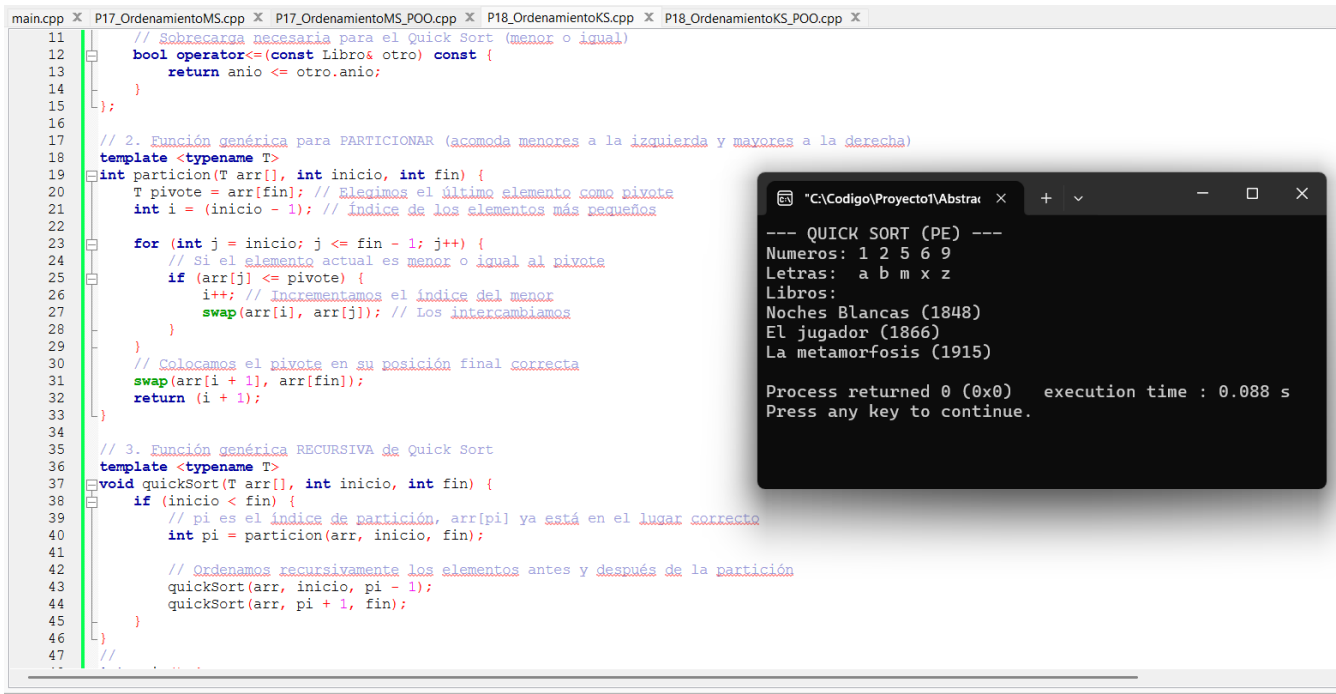
Process returned 0 (0x0)   execution time : 0.122 s
Press any key to continue.
```

Practica No. 18

En esta práctica vamos a acomodar 3 tipos de datos: caracteres, enteros y un nuevo tipo de dato (Libros), utilizando el algoritmo de ordenamiento Quick Sort (ordenamiento rápido). Al igual que en las prácticas anteriores, los datos los damos definidos en el programa para ejemplificar su funcionamiento.

Para este ordenamiento, que también utiliza la estrategia de "Divide y Vencerás", implementamos una lógica basada en un "Pivote". El algoritmo selecciona un elemento del arreglo (en nuestro código, el último) y recorre los demás elementos comparándolos con él. Su objetivo principal es particionar el arreglo: todos los elementos menores o iguales al pivote se intercambian para moverse a su izquierda, y todos los mayores se quedan a su derecha. Al terminar este ciclo, el pivote se coloca exactamente en su posición final ordenada.

Una vez que el pivote original está en su lugar, el algoritmo utiliza la recursividad para llamarse a sí mismo y repetir exactamente el mismo proceso de elegir un nuevo pivote para el sub-arreglo izquierdo y otro para el sub-arreglo derecho. Esto se repite hasta que todos los elementos quedan acomodados.



```
main.cpp x P17_OrdenamientoMS.cpp x P17_OrdenamientoMS_POO.cpp x P18_OrdenamientoKS.cpp x P18_OrdenamientoKS_POO.cpp x
11 // Sobrecarga necesaria para el Quick Sort (menor o igual)
12 bool operator<=(const Libro& otro) const {
13     return anio <= otro.anio;
14 }
15
16
17 // 2. Función genérica para PARTICIONAR (acomoda menores a la izquierda y mayores a la derecha)
18 template <typename T>
19 int particion(T arr[], int inicio, int fin) {
20     T pivote = arr[fin]; // Elegimos el último elemento como pivote
21     int i = (inicio - 1); // Índice de los elementos más pequeños
22
23     for (int j = inicio; j <= fin - 1; j++) {
24         // Si el elemento actual es menor o igual al pivote
25         if (arr[j] <= pivote) {
26             i++; // Incrementamos el índice del menor
27             swap(arr[i], arr[j]); // Los intercambiamos
28         }
29     }
30     // Colocamos el pivote en su posición final correcta
31     swap(arr[i + 1], arr[fin]);
32     return (i + 1);
33 }
34
35 // 3. Función genérica RECURSIVA de Quick Sort
36 template <typename T>
37 void quickSort(T arr[], int inicio, int fin) {
38     if (inicio < fin) {
39         // pi es el índice de partición, arr[pi] ya está en el lugar correcto
40         int pi = particion(arr, inicio, fin);
41
42         // Ordenamos recursivamente los elementos antes y después de la partición
43         quickSort(arr, inicio, pi - 1);
44         quickSort(arr, pi + 1, fin);
45     }
46 }
47
48 //
```

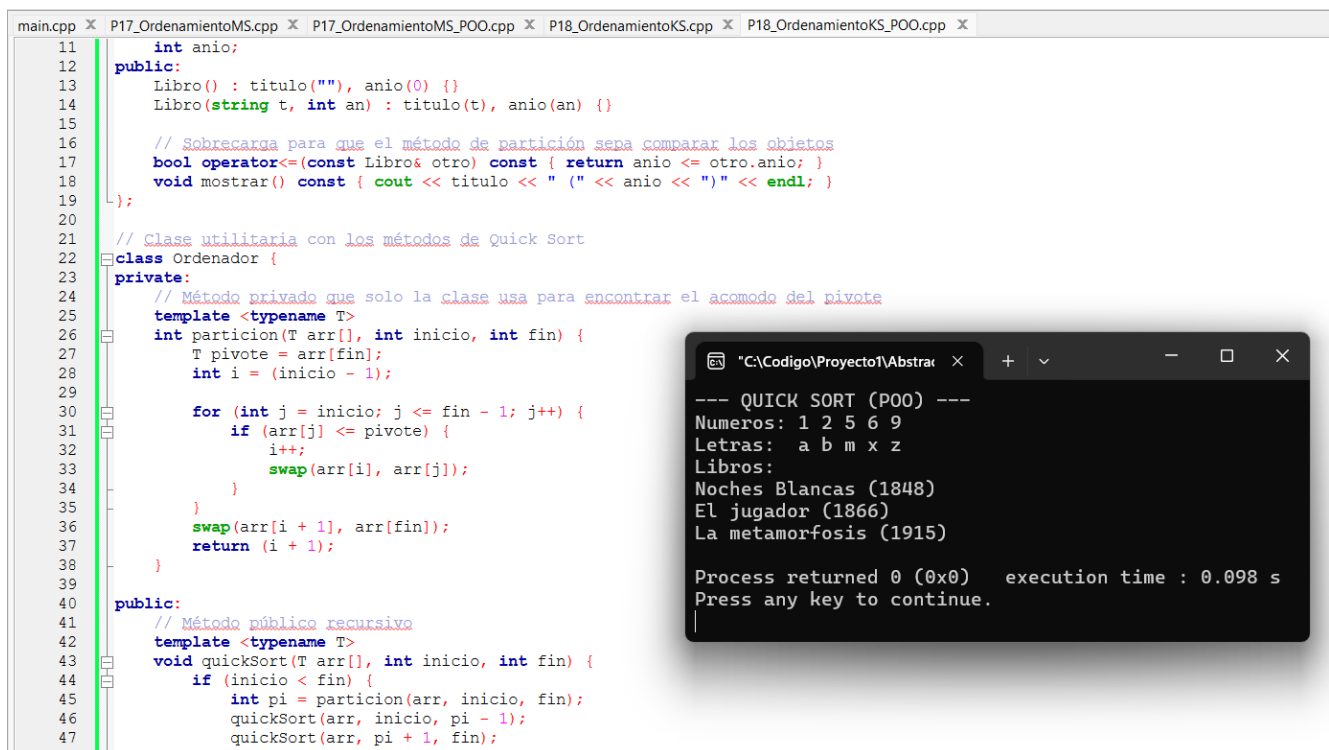
```
--- QUICK SORT (PE) ---
Numeros: 1 2 5 6 9
Letras: a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)

Process returned 0 (0x0)   execution time : 0.088 s
Press any key to continue.
```

A diferencia del método de burbuja, e incluso destacando sobre el Merge Sort en la administración de la memoria (ya que el Quick Sort realiza los intercambios directamente en el mismo arreglo original sin necesidad de crear arreglos temporales), este algoritmo es sumamente eficiente y es considerado uno de los más rápidos en la práctica para manejar grandes volúmenes de información.

También demostramos su funcionamiento en Programación Estructurada y Orientada a Objetos. En ambos paradigmas utilizamos plantillas para que una sola función o método pudiera ordenar los tres tipos de datos. Para el nuevo tipo de dato encapsulado en la estructura y en la clase, aplicamos nuevamente la sobrecarga del operador `<=` para enseñarle a la función de partición cómo debe comparar los años de los libros contra el pivote en turno.

Finalmente, como hemos venido trabajando en el proyecto, integramos este nuevo método de ordenamiento como opciones adicionales dentro del menú de nuestro archivo main principal, manteniendo nuestro programa modular y fácil de ejecutar.



The image shows a C++ IDE with several files open: `main.cpp`, `P17_OrdenamientoMS.cpp`, `P17_OrdenamientoMS_POO.cpp`, `P18_OrdenamientoKS.cpp`, and `P18_OrdenamientoKS_POO.cpp`. The active file is `main.cpp`, which contains the following code:

```
11 int anio;
12 public:
13     Libro() : titulo(""), anio(0) {}
14     Libro(string t, int an) : titulo(t), anio(an) {}
15
16     // Sobrecarga para que el método de partición sepa comparar los objetos
17     bool operator<=(const Libro& otro) const { return anio <= otro.anio; }
18     void mostrar() const { cout << titulo << " (" << anio << ")" << endl; }
19 };
20
21 // Clase utilitaria con los métodos de Quick Sort
22 class Ordenador {
23 private:
24     // Método privado que solo la clase usa para encontrar el acomodo del pivote
25     template <typename T>
26     int particion(T arr[], int inicio, int fin) {
27         T pivote = arr[fin];
28         int i = (inicio - 1);
29
30         for (int j = inicio; j <= fin - 1; j++) {
31             if (arr[j] <= pivote) {
32                 i++;
33                 swap(arr[i], arr[j]);
34             }
35             swap(arr[i + 1], arr[fin]);
36             return (i + 1);
37         }
38     }
39
40 public:
41     // Método público recursivo
42     template <typename T>
43     void quickSort(T arr[], int inicio, int fin) {
44         if (inicio < fin) {
45             int pi = particion(arr, inicio, fin);
46             quickSort(arr, inicio, pi - 1);
47             quickSort(arr, pi + 1, fin);
48         }
49     }
50 }
```

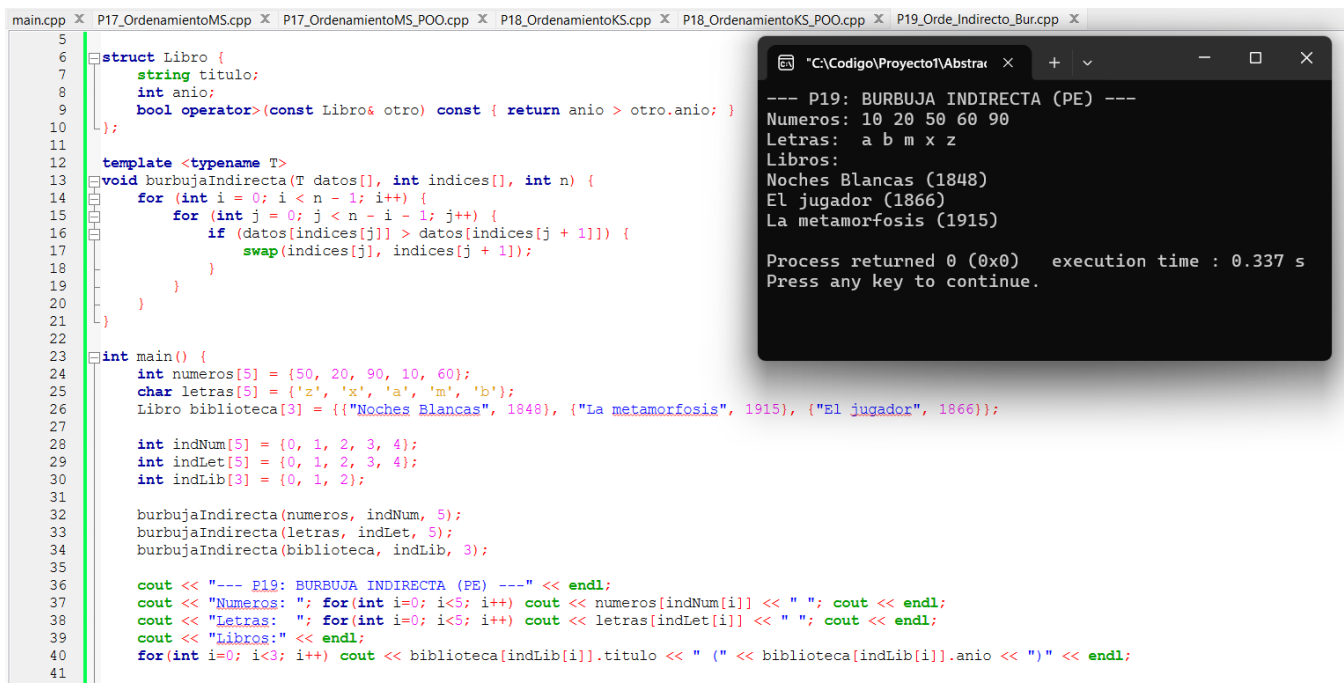
The output window shows the following text:

```
--- QUICK SORT (POO) ---
Numeros: 1 2 5 6 9
Letras:  a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)

Process returned 0 (0x0)   execution time : 0.098 s
Press any key to continue.
```

Practica No. 19

En esta práctica implementamos el ordenamiento de Burbuja para acomodar 3 tipos de datos: caracteres, enteros y nuestro nuevo tipo de dato (Libros), pero esta vez utilizando la técnica de ordenamiento indirecto en Programación Estructurada (PE). La lógica principal de esta técnica consiste en no mover los datos originales de su lugar en la memoria, ya que, en un escenario real con objetos muy pesados, intercambiarlos consumiría demasiado tiempo y recursos. En su lugar, creamos un arreglo adicional llamado "índices". El algoritmo de burbuja sigue comparando los datos originales para saber cuál es mayor, pero al momento de hacer el intercambio (swap), únicamente intercambia los números del arreglo de índices. Al final, leemos los datos originales guiándonos por este arreglo de índices ya ordenado. Implementamos esta lógica utilizando funciones genéricas con plantillas, pasando como parámetros tanto el arreglo de datos como el de índices y manteniendo toda la ejecución secuencial dentro de un mismo archivo.

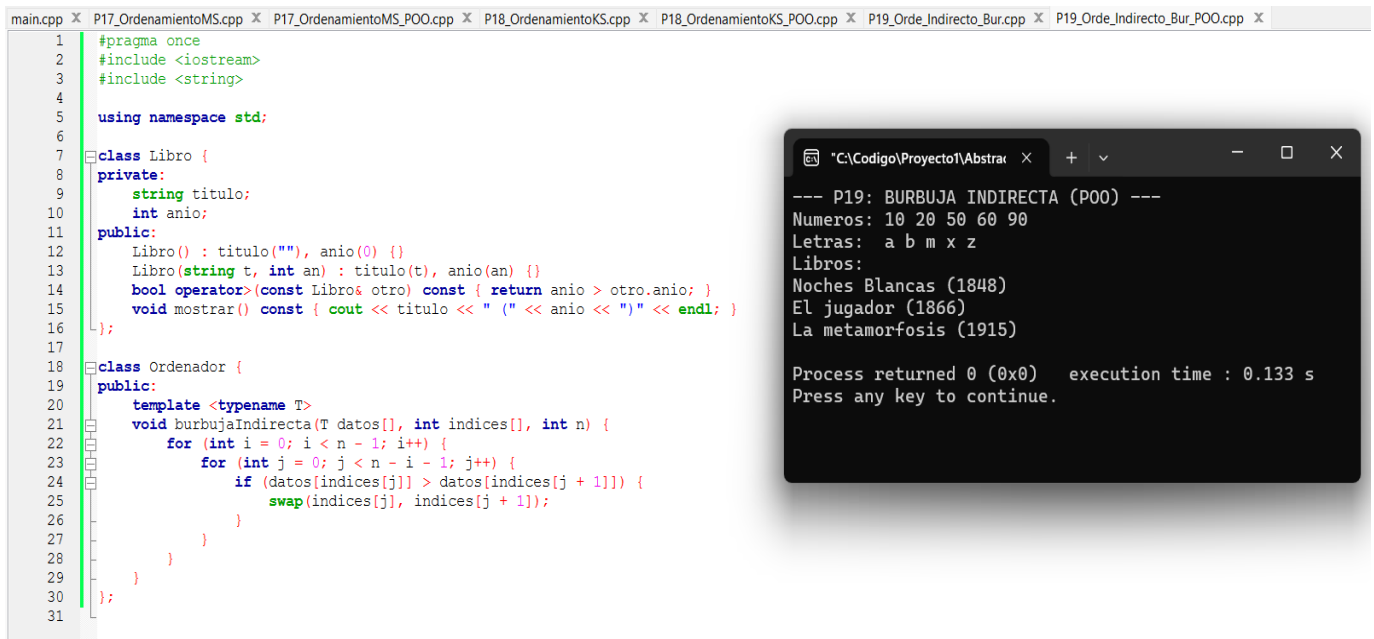


```
5
6 struct Libro {
7     string titulo;
8     int anio;
9     bool operator>(const Libro& otro) const { return anio > otro.anio; }
10 };
11
12 template <typename T>
13 void burbujaIndirecta(T datos[], int indices[], int n) {
14     for (int i = 0; i < n - 1; i++) {
15         for (int j = 0; j < n - i - 1; j++) {
16             if (datos[indices[j]] > datos[indices[j + 1]]) {
17                 swap(indices[j], indices[j + 1]);
18             }
19         }
20     }
21 }
22
23 int main() {
24     int numeros[5] = {50, 20, 90, 10, 60};
25     char letras[5] = {'z', 'x', 'a', 'm', 'b'};
26     Libro biblioteca[3] = {"Noches Blancas", 1848}, {"La metamorfosis", 1915}, {"El jugador", 1866};
27
28     int indNum[5] = {0, 1, 2, 3, 4};
29     int indLet[5] = {0, 1, 2, 3, 4};
30     int indLib[3] = {0, 1, 2};
31
32     burbujaIndirecta(numeros, indNum, 5);
33     burbujaIndirecta(letras, indLet, 5);
34     burbujaIndirecta(biblioteca, indLib, 3);
35
36     cout << "--- P19: BURBUJA INDIRECTA (PE) ---" << endl;
37     cout << "Numeros: "; for(int i=0; i<5; i++) cout << numeros[indNum[i]] << " "; cout << endl;
38     cout << "Letras: "; for(int i=0; i<5; i++) cout << letras[indLet[i]] << " "; cout << endl;
39     cout << "Libros: " << endl;
40     for(int i=0; i<3; i++) cout << biblioteca[indLib[i]].titulo << " (" << biblioteca[indLib[i]].anio << ")" << endl;
41 }
```

```
--- P19: BURBUJA INDIRECTA (PE) ---
Numeros: 10 20 50 60 90
Letras:  a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)

Process returned 0 (0x0)   execution time : 0.337 s
Press any key to continue.
```

Por otro lado, para la versión en Programación Orientada a Objetos (POO), llevamos esta misma lógica de ordenamiento indirecto a un nivel mayor de abstracción. Encapsulamos el método genérico de burbuja dentro de nuestra clase Ordenador. La clase Libro mantuvo su sobrecarga del operador mayor que para saber compararse a sí misma. Esto demuestra que a la clase ordenadora no le importa qué tipo de dato está evaluando en los índices, delegando la responsabilidad de la comparación al objeto mismo y manteniendo el código del archivo principal mucho más limpio y protegido.



```
1  #pragma once
2  #include <iostream>
3  #include <string>
4
5  using namespace std;
6
7  class Libro {
8  private:
9      string titulo;
10     int anio;
11 public:
12     Libro() : titulo(""), anio(0) {}
13     Libro(string t, int an) : titulo(t), anio(an) {}
14     bool operator>(const Libro& otro) const { return anio > otro.anio; }
15     void mostrar() const { cout << titulo << " (" << anio << ")" << endl; }
16 };
17
18 class Ordenador {
19 public:
20     template <typename T>
21     void burbujaIndirecta(T datos[], int indices[], int n) {
22         for (int i = 0; i < n - 1; i++) {
23             for (int j = 0; j < n - i - 1; j++) {
24                 if (datos[indices[j]] > datos[indices[j + 1]]) {
25                     swap(indices[j], indices[j + 1]);
26                 }
27             }
28         }
29     }
30 };
31
```

```
--- P19: BURBUJA INDIRECTA (POO) ---
Numeros: 10 20 50 60 90
Letras:  a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)

Process returned 0 (0x0)   execution time : 0.133 s
Press any key to continue.
```

Practica No. 20

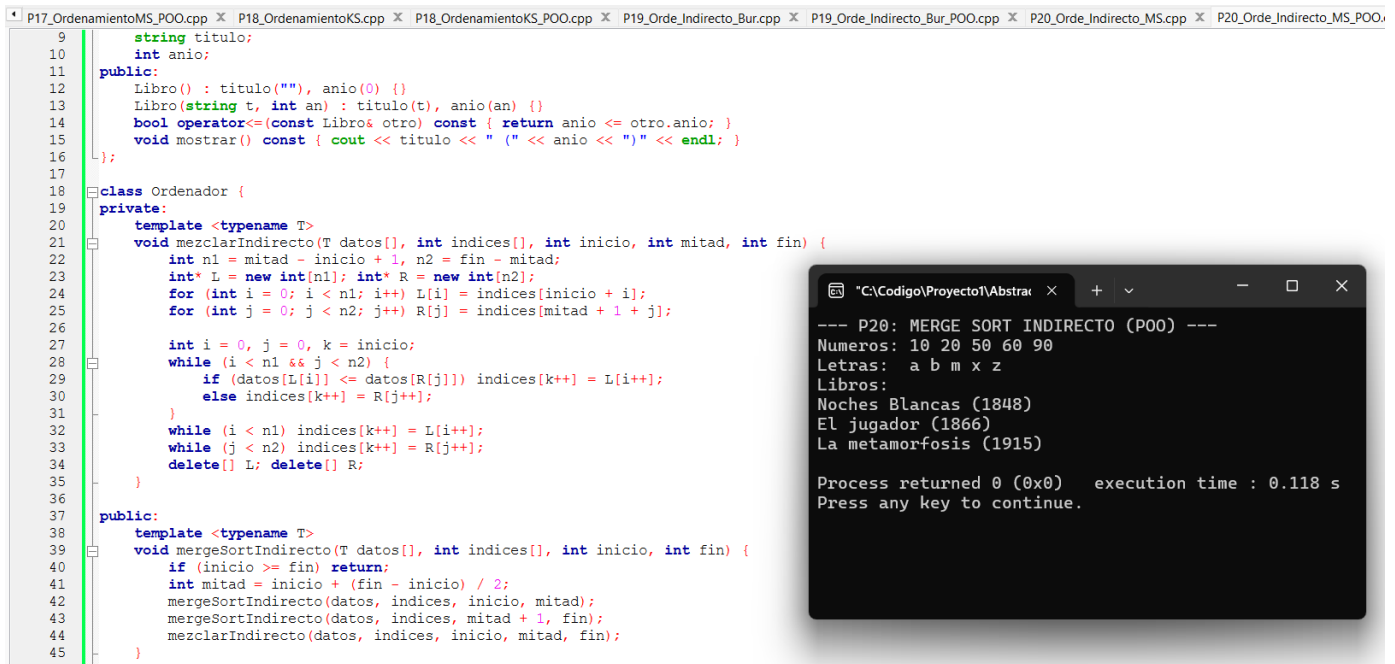
Para esta práctica aplicamos el algoritmo Merge Sort utilizando nuevamente la técnica de ordenamiento indirecto para nuestros 3 conjuntos de datos bajo el paradigma de Programación Estructurada. Mantuvimos la estrategia de "Divide y Vencerás", partiendo el arreglo a la mitad de forma recursiva. Sin embargo, el cambio fundamental es que las divisiones y las fusiones ocurren exclusivamente sobre el arreglo de índices. Durante la fase de mezcla, el algoritmo compara los valores originales y acomoda los índices ganadores en un arreglo temporal. Las funciones recursivas de división y la función de fusión se manejaron de manera global en un solo archivo. Al usar estructuras, sobrecargamos el operador menor o igual para permitir que la plantilla genérica procesara los libros sin alterar la lógica de los enteros o caracteres.

```
main.cpp x P17_OrdenamientoMS.cpp x P17_OrdenamientoMS_POO.cpp x P18_OrdenamientoKS.cpp x P18_OrdenamientoKS_POO.cpp x P19_Orde_Indirecto_Bur.cpp x P19_Orde_Indirecto_Bur_POO.cpp x P20_Orde_Indirecto_Mt
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 struct Libro {
7     string titulo;
8     int anio;
9     bool operator<=(const Libro& otro) const { return anio <= otro.anio; }
10 };
11
12 template <typename T>
13 void mezclarIndirecto(T datos[], int indices[], int inicio, int mitad, int fin) {
14     int n1 = mitad - inicio + 1, n2 = fin - mitad;
15     int* L = new int[n1]; int* R = new int[n2];
16     for (int i = 0; i < n1; i++) L[i] = indices[inicio + i];
17     for (int j = 0; j < n2; j++) R[j] = indices[mitad + 1 + j];
18
19     int i = 0, j = 0, k = inicio;
20     while (i < n1 && j < n2) {
21         if (datos[L[i]] <= datos[R[j]]) indices[k++] = L[i++];
22         else indices[k++] = R[j++];
23     }
24     while (i < n1) indices[k++] = L[i++];
25     while (j < n2) indices[k++] = R[j++];
26     delete[] L; delete[] R;
27 }
28
29 template <typename T>
30 void mergeSortIndirecto(T datos[], int indices[], int inicio, int fin) {
31     if (inicio >= fin) return;
32     int mitad = inicio + (fin - inicio) / 2;
33     mergeSortIndirecto(datos, indices, inicio, mitad);
34     mergeSortIndirecto(datos, indices, mitad + 1, fin);
35     mezclarIndirecto(datos, indices, inicio, mitad, fin);
36 }
37
```

```
--- P20: MERGE SORT INDIRECTO (PE) ---
Numeros: 10 20 50 60 90
Letras: a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)

Process returned 0 (0x0)   execution time : 0.119 s
Press any key to continue.
```

En la implementación de Programación Orientada a Objetos para el Merge Sort indirecto, el enfoque principal fue proteger la lógica interna del algoritmo. Abstrajimos la complejidad ocultando la función de fusión, encargada de mezclar los índices, como un método privado de la clase Ordenador. De esta forma, dejamos público únicamente el método recursivo principal que recibe los arreglos. Esto asegura que la manipulación de la memoria dinámica y los arreglos temporales de índices ocurra de manera segura dentro del objeto, haciendo que su uso desde el main sea mucho más intuitivo y modular.

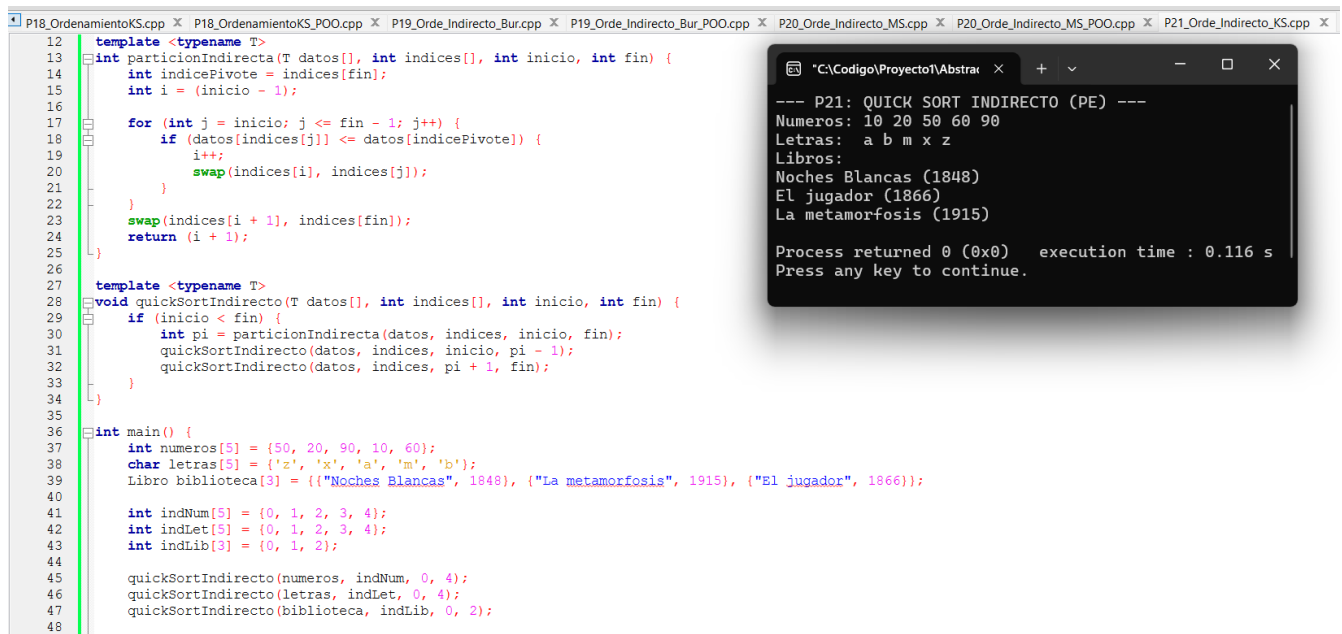


```
9      string titulo;  
10     int anio;  
11 public:  
12     Libro() : titulo(""), anio(0) {}  
13     Libro(string t, int an) : titulo(t), anio(an) {}  
14     bool operator<=(const Libro& otro) const { return anio <= otro.anio; }  
15     void mostrar() const { cout << titulo << " (" << anio << ")" << endl; }  
16 };  
17  
18 class Ordenador {  
19 private:  
20     template <typename T>  
21     void mezclarIndirecto(T datos[], int indices[], int inicio, int mitad, int fin) {  
22         int n1 = mitad - inicio + 1, n2 = fin - mitad;  
23         int* L = new int[n1]; int* R = new int[n2];  
24         for (int i = 0; i < n1; i++) L[i] = indices[inicio + i];  
25         for (int j = 0; j < n2; j++) R[j] = indices[mitad + 1 + j];  
26  
27         int i = 0, j = 0, k = inicio;  
28         while (i < n1 && j < n2) {  
29             if (datos[L[i]] <= datos[R[j]]) indices[k++] = L[i++];  
30             else indices[k++] = R[j++];  
31         }  
32         while (i < n1) indices[k++] = L[i++];  
33         while (j < n2) indices[k++] = R[j++];  
34         delete[] L; delete[] R;  
35     }  
36  
37 public:  
38     template <typename T>  
39     void mergeSortIndirecto(T datos[], int indices[], int inicio, int fin) {  
40         if (inicio >= fin) return;  
41         int mitad = inicio + (fin - inicio) / 2;  
42         mergeSortIndirecto(datos, indices, inicio, mitad);  
43         mergeSortIndirecto(datos, indices, mitad + 1, fin);  
44         mezclarIndirecto(datos, indices, inicio, mitad, fin);  
45     }  
46 }
```

```
--- P20: MERGE SORT INDIRECTO (P00) ---  
Numeros: 10 20 50 60 90  
Letras:  a b m x z  
Libros:  
Noches Blancas (1848)  
El jugador (1866)  
La metamorfosis (1915)  
  
Process returned 0 (0x0)   execution time : 0.118 s  
Press any key to continue.
```


Practica No. 21

En esta última práctica implementamos el algoritmo Quick Sort de manera indirecta utilizando Programación Estructurada. La adaptación al ordenamiento indirecto consistió en que el "pivote" seleccionado es un elemento del arreglo de índices. Al recorrer el arreglo, comparamos los datos originales apuntados por los índices actuales contra el dato apuntado por el índice pivote. Si el dato es menor o igual, intercambiamos de lugar únicamente los índices, dejando intactos los arreglos originales de números, letras y libros. El algoritmo se implementó mediante el paso de los arreglos a funciones sueltas y genéricas, realizando la gestión de los índices de manera directa y procedimental en un solo archivo, lo que muestra cómo se manipula la memoria de forma transparente.

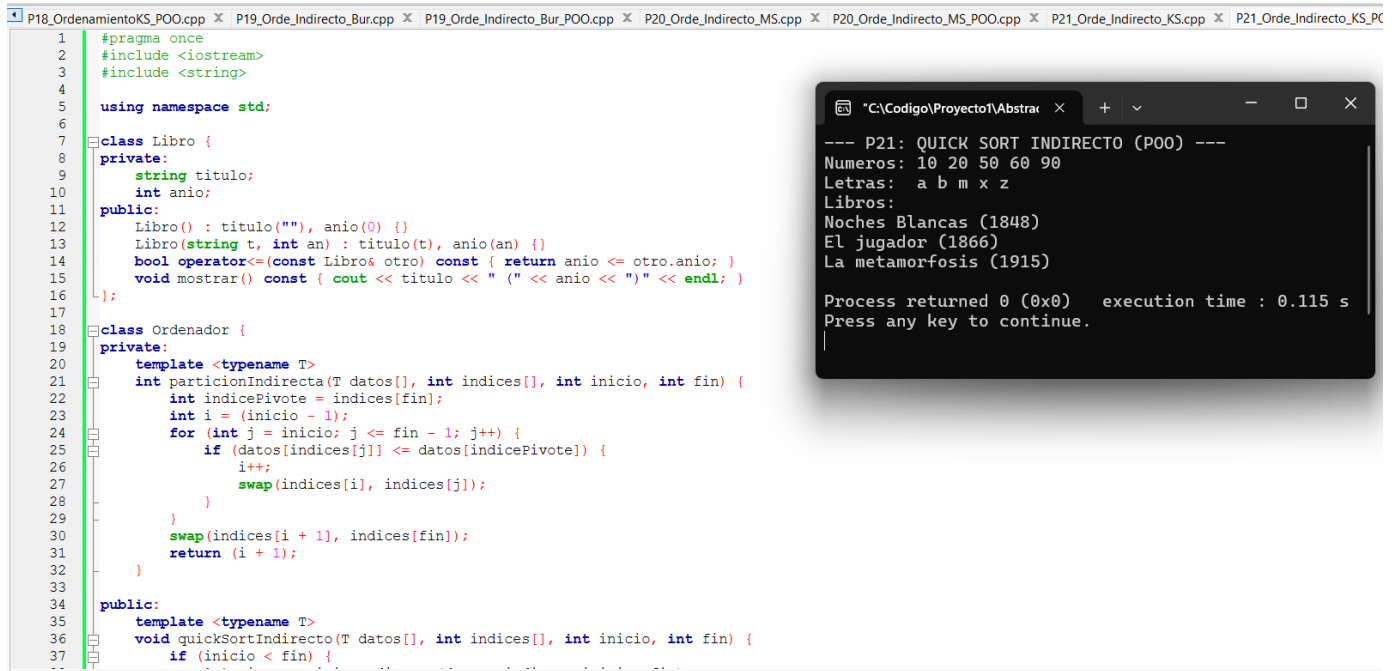


```
12 template <typename T>
13 int particionIndirecta(T datos[], int indices[], int inicio, int fin) {
14     int indicePivote = indices[fin];
15     int i = (inicio - 1);
16
17     for (int j = inicio; j <= fin - 1; j++) {
18         if (datos[indices[j]] <= datos[indicePivote]) {
19             i++;
20             swap(indices[i], indices[j]);
21         }
22     }
23     swap(indices[i + 1], indices[fin]);
24     return (i + 1);
25 }
26
27 template <typename T>
28 void quickSortIndirecto(T datos[], int indices[], int inicio, int fin) {
29     if (inicio < fin) {
30         int pi = particionIndirecta(datos, indices, inicio, fin);
31         quickSortIndirecto(datos, indices, inicio, pi - 1);
32         quickSortIndirecto(datos, indices, pi + 1, fin);
33     }
34 }
35
36 int main() {
37     int numeros[5] = {50, 20, 90, 10, 60};
38     char letras[5] = {'z', 'x', 'a', 'm', 'b'};
39     Libro biblioteca[3] = {"Noches Blancas", 1848}, {"La metamorfosis", 1915}, {"El jugador", 1866};
40
41     int indNum[5] = {0, 1, 2, 3, 4};
42     int indLet[5] = {0, 1, 2, 3, 4};
43     int indLib[3] = {0, 1, 2};
44
45     quickSortIndirecto(numeros, indNum, 0, 4);
46     quickSortIndirecto(letras, indLet, 0, 4);
47     quickSortIndirecto(biblioteca, indLib, 0, 2);
48 }
```

```
--- P21: QUICK SORT INDIRECTO (PE) ---
Numeros: 10 20 50 60 90
Letras:  a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)

Process returned 0 (0x0)   execution time : 0.116 s
Press any key to continue.
```

Finalmente, para la versión en Programación Orientada a Objetos (POO) del Quick Sort indirecto, separamos el código en dos archivos para mejorar la organización del proyecto. La clase Ordenador se encargó de realizar la partición de los índices y las llamadas recursivas de forma completamente interna y encapsulada. Por su parte, la clase Libro resolvió las comparaciones complejas mediante polimorfismo gracias a la sobrecarga del operador menor o igual. El resultado es un programa altamente modular y escalable, donde el ordenamiento rápido opera de forma indirecta sin comprometer la estructura original de los datos ni la seguridad de la información.



```
1  #pragma once
2  #include <iostream>
3  #include <string>
4
5  using namespace std;
6
7  class Libro {
8  private:
9      string titulo;
10     int anio;
11 public:
12     Libro() : titulo(""), anio(0) {}
13     Libro(string t, int an) : titulo(t), anio(an) {}
14     bool operator<=(const Libro& otro) const { return anio <= otro.anio; }
15     void mostrar() const { cout << titulo << " (" << anio << ")" << endl; }
16 };
17
18 class Ordenador {
19 private:
20     template <typename T>
21     int particionIndirecta(T datos[], int indices[], int inicio, int fin) {
22         int indicePivote = indices[fin];
23         int i = (inicio - 1);
24         for (int j = inicio; j <= fin - 1; j++) {
25             if (datos[indices[j]] <= datos[indicePivote]) {
26                 i++;
27                 swap(indices[i], indices[j]);
28             }
29         }
30         swap(indices[i + 1], indices[fin]);
31         return (i + 1);
32     }
33 public:
34     template <typename T>
35     void quickSortIndirecto(T datos[], int indices[], int inicio, int fin) {
36         if (inicio < fin) {
37             int i = particionIndirecta(datos, indices, inicio, fin);
38             quickSortIndirecto(datos, indices, inicio, i - 1);
39             quickSortIndirecto(datos, indices, i + 1, fin);
40         }
41     }
42 };
43
44 int main() {
45     int numeros[] = {10, 20, 50, 60, 90};
46     string letras[] = {"a", "b", "m", "x", "z"};
47     Libro libros[] = {"Noches Blancas", 1848}, {"El jugador", 1866}, {"La metamorfosis", 1915};
48     Ordenador o;
49     o.quickSortIndirecto(numeros, numeros, 0, 4);
50     o.quickSortIndirecto(letras, letras, 0, 4);
51     o.quickSortIndirecto(libros, libros, 0, 2);
52     return 0;
53 }
```

```
--- P21: QUICK SORT INDIRECTO (POO) ---
Numeros: 10 20 50 60 90
Letras:  a b m x z
Libros:
Noches Blancas (1848)
El jugador (1866)
La metamorfosis (1915)

Process returned 0 (0x0)   execution time : 0.115 s
Press any key to continue.
```

Práctica 6 – Pila Dinámica

En esta práctica programé una **pila dinámica** utilizando el concepto **LIFO** (Last In, First Out), lo que significa que el último dato que meto es siempre el primero en salir. A diferencia de la práctica anterior, aquí no usé un arreglo con tamaño fijo, sino que usé **nodos y punteros** para que la pila pueda crecer todo lo que sea necesario.

El programa tiene las funciones básicas:

- **Push (Insertar):** Crea un nuevo nodo en la memoria, le guarda el valor y lo pone hasta arriba de la pila, actualizando el puntero cima.
- **Pop (Eliminar):** Saca el elemento que está en la parte superior. Lo que hace es mover la cima al nodo de abajo y borra el nodo que sacamos para no desperdiciar memoria.
- **Mostrar:** Recorre toda la pila desde la cima hasta el fondo para enseñarme qué datos tiene guardados.

¿Qué fue lo más difícil?

Lo más pesado fue entender bien cómo funcionan los **punteros** para no "romper" la pila. Especialmente en la función push, tienes que conectar el nuevo nodo con el que ya era la cima antes de actualizar la cima, porque si lo haces al revés, pierdes la dirección de todo lo que ya habías guardado y se crea un caos en la memoria.

```
C:\Users\Patlani\Downloads\>
Contenido de la pila:
5
4
3
3
2
1
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\>
Pila vacia
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\>
Saliendo...
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\>
Elemento eliminado: 5
Presione una tecla para continuar . . . |
```

Práctica 7– Cola Dinámica

En esta práctica implementé una **cola dinámica**, que funciona bajo el principio **FIFO** (First In, First Out), es decir, el primer dato que entra es obligatoriamente el primero en salir. A diferencia de la pila, aquí la estructura es como una fila del banco.

El programa gestiona los datos de esta manera:

- **Enqueue (Insertar):** Agrega un elemento al final de la cola. Para que esto sea rápido, el programa usa un puntero llamado fin que sabe exactamente dónde está el último lugar.
- **Dequeue (Eliminar):** Saca al elemento que está al principio de la fila. Aquí se usa el puntero frente para saber a quién le toca salir y luego se libera ese espacio de memoria.
- **Mostrar:** Recorre todos los nodos desde el frente hasta el fin para enseñarme el orden actual de la cola.

¿Qué fue lo más difícil?

Lo más complicado fue **gestionar dos punteros al mismo tiempo** (frente y fin) y mantenerlos sincronizados. Por ejemplo, cuando la cola está vacía y metes el primer elemento, tienes que hacer que tanto frente como fin apunten a ese mismo nodo. Lo mismo pasa al eliminar: si sacas el último elemento que quedaba, tienes que acordarte de resetear ambos punteros a NULL, porque si no, el programa piensa que todavía hay algo ahí y truenas cuando intentas hacer otra operación

```
C:\Users\Patlani\Downloads\F x + v
Contenido de la cola:
1 2 3 4 5
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F x + v
Elemento eliminado: 5
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F x + v
Cola vacia
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F x + v
Elemento eliminado: 1
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F x + v
Saliendo del programa...
Presione una tecla para continuar . . . |
```

Práctica 8– Pila

En esta práctica volví a trabajar con una **Pila (LIFO)**, pero esta vez de una forma mucho más avanzada y profesional: utilicé la **STL (Standard Template Library)** de C++. En lugar de estar creando nodos, clases para el puntero cima y preocuparme por el new o el delete, simplemente incluí la librería <stack>.

El programa permite hacer lo mismo que antes pero de forma más eficiente:

Push: Inserta un elemento al tope de la pila usando la función directa de la librería.

Pop: Elimina el elemento de arriba. Aquí es más seguro porque la librería ya maneja internamente la memoria.

Ver Cima (Top): Me permite observar qué valor está hasta arriba sin necesidad de sacarlo, usando la función .top().

Mostrar: Como la pila de la STL no se puede recorrer fácilmente sin vaciarla, lo que hice fue crear una **pila auxiliar** (una copia) para ir imprimiendo los valores uno por uno mientras los sacaba de esa copia, así la pila original se queda intacta.

¿Qué fue lo más difícil?

Lo más difícil fue cambiar el "chip" de programar todo a mano a confiar en la librería. Al principio me confundía porque ya no tenía acceso directo a los punteros, y entender que para mostrar los datos tenía que **clonar la pila en una auxiliar** fue un reto de lógica. También tuve que tener cuidado con la función .pop() de la STL, porque a diferencia de mis códigos anteriores, la función .pop() de la librería solo borra el elemento pero no te lo devuelve, así que si quieres saber qué había ahí, primero tienes que usar .top().

```
C:\Users\Patlani\Downloads\F x + v
Contenido de la pila:
5
4
3
2
1
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F x + v
Elemento eliminado: 5
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F x + v
Elemento en la cima: 4
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F x + v
===== PILA STL =====
1. Insertar (Push)
2. Eliminar (Pop)
3. Mostrar
4. Ver cima
5. Salir
Seleccione una opcion: 1|
```

```
C:\Users\Patlani\Downloads\F x + v
Saliendo del programa...
Presione una tecla para continuar . . . |
```

Práctica 9 – Cola

En esta práctica volví a trabajar con una **Cola (FIFO)**, pero elevando el nivel al utilizar la **STL (Standard Template Library)**. En lugar de pelearme con los punteros frente y fin o crear manualmente la lógica de los nodos, simplemente usé la librería `<queue>`.

El programa me permite gestionar la fila de datos de forma muy sencilla:

- **Enqueue (Insertar):** Meto un dato al final de la cola con la función `.push()`. La librería se encarga de que siempre se vaya al último lugar de forma automática.
- **Deque (Eliminar):** Saco al que está al frente usando `.pop()`. Al igual que con la pila, la librería gestiona la memoria por detrás.
- **Ver Frente:** Con la función `.front()` puedo ver quién es el siguiente en salir sin tener que borrarlo todavía.
- **Mostrar:** Como las colas de la STL no se pueden recorrer directamente sin vaciarlas, apliqué el mismo truco que con la pila: hice una **cola auxiliar** (una copia exacta) y fui imprimiendo y sacando los elementos de ahí para no perder mi cola original.

¿Qué fue lo más difícil?

Lo más difícil fue dominar la lógica de **copiar la estructura a una auxiliar para mostrarla**.

```
C:\Users\Patlani\Downloads\F >
Contenido de la cola:
1 2 4 6 8
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Elemento al frente: 2
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Elemento eliminado: 1
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Tamano de la cola: 4
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Cola vacia
Presione una tecla para continuar . . . |
```

Práctica 10 – Lista

En esta práctica utilicé la librería `#include <list>` para manejar una **lista doblemente enlazada** de forma profesional. Al usar la **STL**, el programa se vuelve mucho más potente y versátil que las versiones manuales que hice antes.

Lo que más me costó trabajo fue entender y aplicar los **Iteradores**. En la STL, no puedes usar un simple número o índice para moverte por la lista; tienes que declarar un objeto especial llamado `list<int>::iterator`. Aprender que este iterador funciona como un "puntero inteligente" y que para acceder a su valor dentro de un `for` hay que usar un asterisco (como `*it`) fue la parte más confusa de la práctica. También fue un reto entender que la lista de la STL es doblemente enlazada, lo que significa que el manejo interno es más complejo de lo que parece a simple vista.

```
C:\Users\Patlani\Downloads\F >
Contenido de la lista:
1 3 6 9
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Valor a eliminar: 6
Elemento eliminado (si existia)
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Lista limpiada
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Tamano: 4
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Lista vacia
Presione una tecla para continuar . . . |
```

Práctica 7 Bis

En esta práctica realicé una **lista dinámica** construida desde cero, utilizando nodos y punteros para gestionar la memoria de forma manual. A diferencia de las pilas o colas que tienen reglas muy estrictas, esta lista es mucho más flexible porque me permite interactuar con los datos en cualquier orden.

El programa cuenta con estas funciones principales:

- **Insertar:** Crea un nuevo nodo y lo coloca al inicio de la lista. Lo interesante aquí es que el programa ajusta los punteros para que el nuevo elemento apunte al que antes era el primero, manteniendo toda la cadena unida.
- **Eliminar:** Borra el elemento que está al principio de la lista. Para hacerlo de forma segura, primero guardo la dirección en un nodo temporal, muevo el inicio al siguiente elemento y luego libero la memoria con delete.
- **Buscar:** Esta función recorre la lista nodo por nodo comparando el valor que busco. Si lo encuentra, me dice en qué posición está (empezando desde el 0), lo cual es súper útil para saber dónde están mis datos.
- **Mostrar:** Imprime todos los valores almacenados recorriendo la lista con un nodo auxiliar hasta llegar a NULL.

¿Qué fue lo más difícil?

Lo más complicado fue **la lógica de los punteros en la búsqueda y la inserción**. Como no tengo índices como en un arreglo normal, tuve que aprender a "caminar" por la memoria usando un nodo auxiliar (aux).

```
C:\Users\Patlani\Downloads\F >
Contenido de la lista:
25 20 15 10 5
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Valor a buscar: 15
Elemento encontrado en la posición: 2
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Elemento eliminado: 25
Presione una tecla para continuar . . . |
```

```
C:\Users\Patlani\Downloads\F >
Contenido de la lista:
15 10 5
Presione una tecla para continuar . . . |
```